



MCDI500

Evaluación Sumativa 2 — Fase 2

Notebook de obtención, limpieza y transformación de datos

Informe técnico grupal · 20 % · 63 puntos

Título del proyecto: Pipeline reproducible de preprocesamiento — Calidad del aire para Santiago (ABP, Ciencia de Datos Reproducible)

Integrantes: Rodrigo Chinchón Ayala · Pablo Villalobos González · Sergio Fernández Almonacid

Curso: MCDI500 — Herramientas de Software Científico (202681.2535)

Docente: Dr. Omar Salinas Silva

Institución: Universidad Andrés Bello — Facultad de Ingeniería

Fecha: 08/06/2026

Índice

Índice	2
I. Introducción.....	3
Objetivo general	3
Objetivos específicos.....	3
II. Diseño de soluciones algorítmicas eficientes	4
2.1 Codificación funcional y arquitectura básica del script	4
Flujo operacional.....	5
2.2 Preprocesamiento y transformación del dataset	7
Exploración inicial	7
Limpieza de datos.....	8
Justificación técnica de la imputación con mediana	8
2.3 Transformación de variables.....	8
2.4 Validación técnica y verificación del código	8
III. Notebook ejecutable (F2).....	10
3.1 Evidencias de ejecución por bloque.....	10
3.2 Pipeline completo y persistencia.....	11
3.3 Dependencias utilizadas	11
IV. Repositorio GitHub (F2).....	13
4.1 Estructura del repositorio.....	13
4.2 README técnico actualizado	13
4.3 Control de versiones y contribuciones individuales	13
V. Bibliografía (APA 7)	14
VI. Justificación técnica de las decisiones de preprocesamiento.....	15
6.1 Imputación de valores faltantes mediante mediana	15
6.2 Normalización de variables mediante escalamiento Min-Max	15
6.3 Transformación de variables categóricas mediante One-Hot Encoding	16
6.4 Validación técnica y aseguramiento de calidad	16
6.5 Relación con los objetivos analíticos del proyecto	16
VII. Conclusiones	17
VIII. Anexos	18
Anexo A. Checklist de entrega (rúbrica · 63 pts).....	18
Anexo B. Convención de commits recomendada	18
Anexo C. Comandos de validación sugeridos.....	18
Anexo D. Resumen de defectos del dataset crudo	18

I. Introducción

El presente informe corresponde al segundo avance formal del proyecto ABP de Ciencia de Datos Reproducible del curso MCDI500. Su propósito es documentar la implementación inicial del pipeline de obtención, limpieza, transformación y validación de datos, construido sobre la infraestructura reproducible definida en la Fase 1 y orientado a un caso de uso de análisis de calidad del aire para Santiago.

La problemática abordada es transversal a todo proyecto analítico real: los conjuntos de datos crudos rara vez están listos para el análisis. El *dataset* base utilizado fue diseñado deliberadamente con defectos típicos —registros duplicados, valores faltantes, inconsistencias de formato textual y variables en escalas heterogéneas— de modo que el equipo debe aplicar técnicas rigurosas de preprocesamiento antes de cualquier etapa de exploración o modelado.

El propósito analítico de esta fase es transformar ese conjunto crudo en un dataset consistente, íntegro, trazable y reproducible. Para ello se adopta una arquitectura modular en Python, apoyada en el ecosistema científico (pandas, NumPy y scikit-learn), donde cada etapa del pipeline está encapsulada en funciones especializadas con responsabilidades bien delimitadas.

Objetivo general

Implementar un pipeline reproducible de preprocesamiento de datos que permita obtener un conjunto consistente, íntegro y preparado para las etapas posteriores de análisis exploratorio y modelado analítico, garantizando la trazabilidad de cada transformación aplicada.

Objetivos específicos

- Realizar la obtención y el perfilamiento inicial del dataset (tipos, nulos, duplicados y cardinalidad).
- Identificar y corregir problemas de calidad de datos mediante criterios técnicos justificados.
- Gestionar adecuadamente los valores faltantes en variables numéricas y categóricas.
- Aplicar las transformaciones y normalizaciones necesarias (casting, MinMax, One-Hot Encoding).
- Validar la consistencia e integridad del dataset resultante bajo casos normales, límite y de excepción.
- Documentar técnicamente cada etapa del pipeline y garantizar su reproducibilidad mediante notebook y control de versiones.

La relación entre el conjunto de datos, la estrategia de preprocesamiento y los resultados esperados es directa: a partir de un *dataset* de 21 filas y 9 columnas con nulos y un duplicado, el pipeline produce un dataset depurado de 20 filas, y posteriormente un dataset transformado de 20 filas × 27 columnas, sin nulos ni duplicados, escalado y codificado. De este modo, los procedimientos implementados fortalecen la **reproducibilidad** (ejecución íntegra vía «Restart & Run All»), la **trazabilidad** (cada decisión queda registrada en celdas Markdown y en el historial de versiones) y la **consistencia metodológica** del pipeline frente a la Fase 1.

II. Diseño de soluciones algorítmicas eficientes

Este apartado presenta el desarrollo técnico del sistema, evidenciando la construcción de soluciones algorítmicas eficientes, modulares y bien estructuradas. Cada componente descrito a continuación corresponde a un criterio de la rúbrica.

2.1 Codificación funcional y arquitectura básica del script

La solución adopta una **arquitectura modular** basada en funciones especializadas alojadas en `F2/src/preprocessing.py`, siguiendo principios de mantenibilidad, reutilización y separación de responsabilidades. El *notebook* orquesta estas funciones sin contener lógica de negocio, lo que mantiene el flujo de datos verificable y el código limpio y fácil de interpretar.

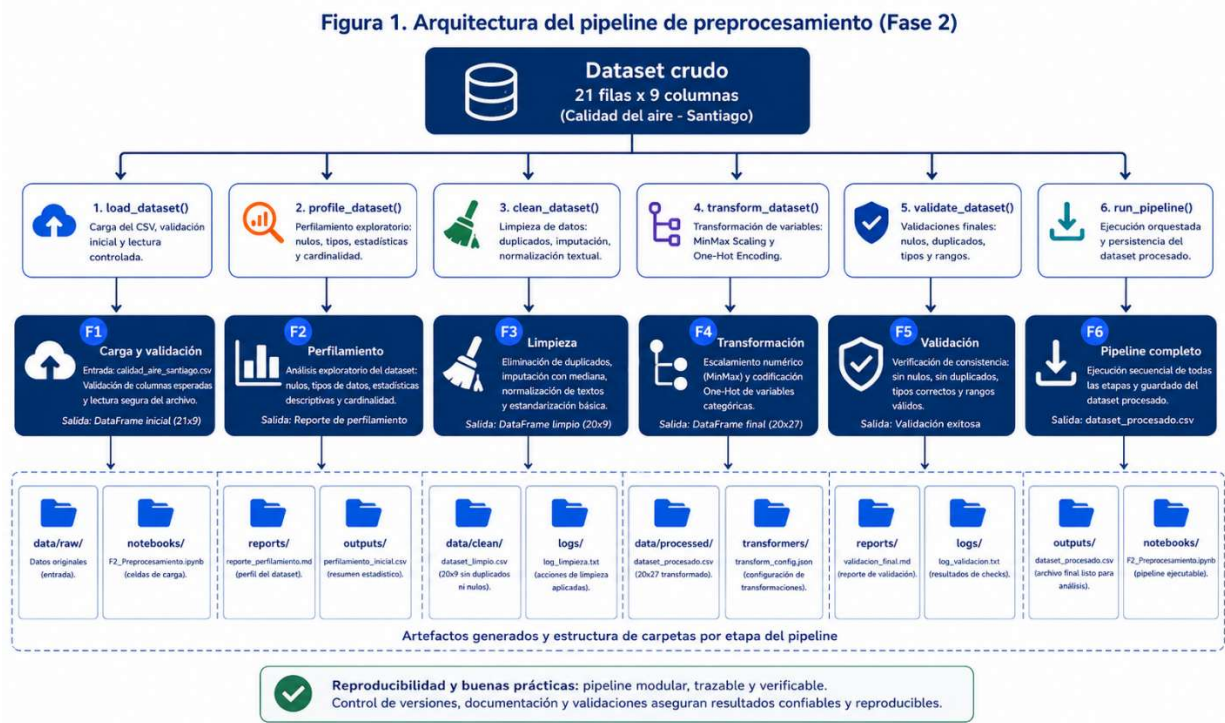
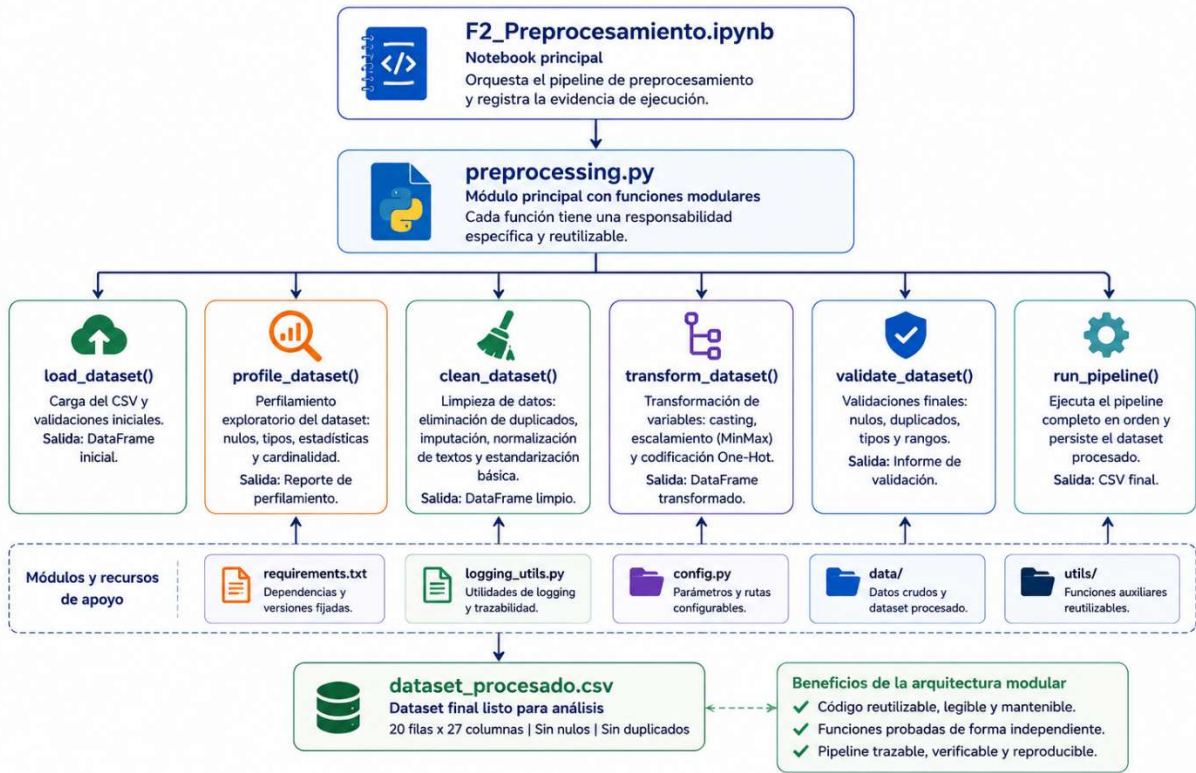


Figura 3. Arquitectura modular del código (Fase 2)



Las funciones principales del pipeline son:

Función	Responsabilidad
load_dataset()	Carga del CSV crudo y validación inicial (existencia del archivo, lectura controlada, casting de tipos básicos).
profile_dataset()	Perfilamiento y diagnóstico: tipo, nº de nulos, porcentaje de nulos y cardinalidad (únicos) por variable.
clean_dataset()	Limpieza: eliminación de duplicados, casting, imputación numérica con mediana, normalización textual y relleno de categóricas no informadas.
transform_dataset()	Transformación: normalización MinMax de variables numéricas y One-Hot Encoding de variables nominales.
validate_dataset()	Verificación de calidad: filas, columnas, duplicados, nulos totales y banderas booleanas sin_nulos / sin_duplicados.
run_pipeline()	Orquestación integral del flujo y persistencia del dataset procesado en disco.

Tabla 1. Funciones del pipeline y sus responsabilidades.

Flujo operacional



Figura 1. Flujo operacional del pipeline de preprocesamiento.

La celda de arranque del notebook resuelve la raíz del repositorio de forma robusta (independiente del directorio de ejecución), incorpora el módulo de funciones al path e imprime el entorno reproducible verificado:

```
import sys
from pathlib import Path
import pandas as pd
from IPython.display import display
ROOT = Path.cwd().resolve()
if ROOT.name == 'notebooks':
    ROOT = ROOT.parents[1]
sys.path.append(str(ROOT / 'F2' / 'src'))
from preprocessing import (
    load_dataset, profile_dataset, clean_dataset,
    transform_dataset, validate_dataset, run_pipeline,
)
RAW = ROOT / 'F2' / 'data' / 'raw' / 'dataset_base.csv'
OUT = ROOT / 'F2' / 'data' / 'processed' / 'dataset_procesado.csv'
```

Listado 1. Inicialización reproducible del entorno y carga del módulo de funciones.

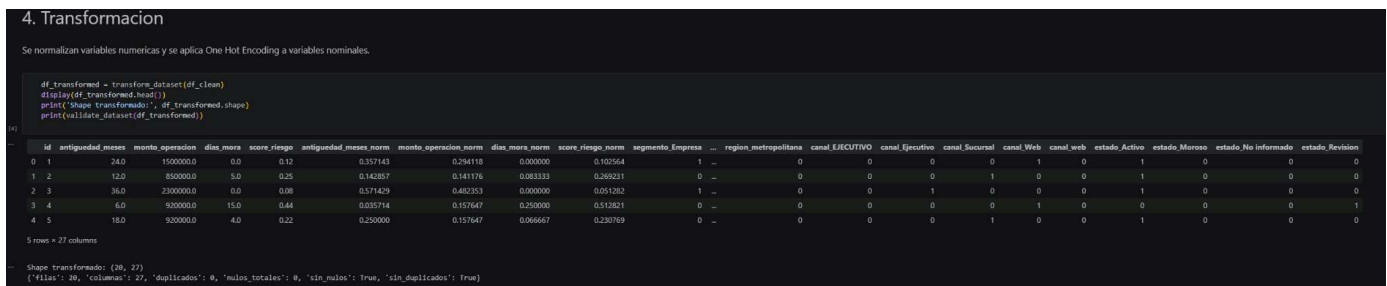


Figura 2. Ejecución de la celda de inicialización: Python 3.12.10, Pandas 3.0.3 y ruta del dataset verificada.

2.2 Preprocesamiento y transformación del dataset

El pipeline implementa una estrategia completa de preparación de datos compuesta por exploración inicial, limpieza y transformación.

Figura 2. Ciclo de calidad de datos (Fase 2)



Exploración inicial

Durante el perfilamiento se analizan los tipos de datos, la distribución de variables, los valores nulos, los registros duplicados, la cardinalidad de las variables categóricas y la consistencia estructural. El dataset crudo presenta 21 filas × 9 columnas, con un registro duplicado y nulos en cinco variables.

Limpieza de datos

- **Eliminación de duplicados:** se detecta y elimina el registro repetido (id = 12) para evitar sesgos en análisis posteriores.
- **Valores faltantes (numéricas):** imputación mediante la mediana de cada variable.
- **Valores faltantes (categóricas):** imputación mediante la categoría «No informado».
- **Normalización textual:** se corrigen inconsistencias de mayúsculas/minúsculas y espacios (p. ej. « pyme », « metropolitana », « web », « EJECUTIVO »).

Justificación técnica de la imputación con mediana

La mediana se selecciona por su robustez frente a valores extremos: reduce las distorsiones estadísticas y mantiene la tendencia central real de cada variable sin verse arrastrada por picos puntuales, evitando introducir un sesgo artificial en los datos.

2.3 Transformación de variables

Casting de tipos: se homogeneizan los tipos de datos para asegurar consistencia operacional antes del escalamiento.

Normalización Min-Max: se escala cada variable numérica al rango [0, 1], homogeneizando rangos, mejorando el comportamiento de los algoritmos posteriores y reduciendo el sesgo por magnitud.

One-Hot Encoding: las variables categóricas nominales se transforman en variables binarias independientes, lo que evita relaciones ordinales artificiales, facilita el procesamiento por algoritmos de Machine Learning y preserva la información categórica original.

2.4 Validación técnica y verificación del código

La validación constituye una etapa fundamental del pipeline y se ejecuta de forma sistemática sobre tres escenarios:

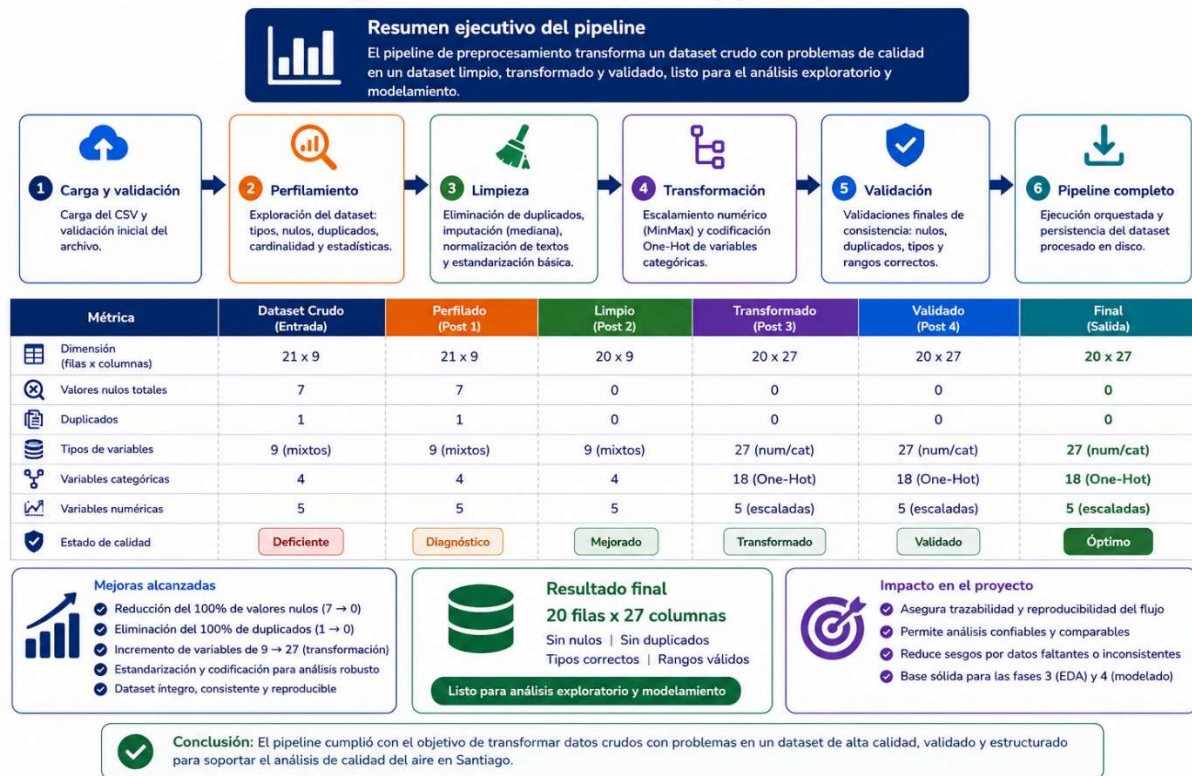
- **Casos normales:** carga correcta del dataset, transformaciones exitosas y generación del dataset procesado.
- **Casos límite:** registros con valores faltantes, variables de baja cardinalidad y registros duplicados.
- **Manejo de excepciones:** controles para archivos inexistentes, datos corruptos, formatos inválidos y errores de lectura.

Las verificaciones formales aplicadas y sus resultados esperados se resumen a continuación:

Validación	Resultado esperado
Valores nulos	0
Registros duplicados	0
Integridad estructural	Correcta
Consistencia de tipos	Correcta
Persistencia del archivo	Exitosa

Tabla 2. Matriz de validación técnica del pipeline.

Figura 4. Matriz de resultados del pipeline (Fase 2)



La función `validate_dataset()` devuelve un diccionario trazable que se incluye como aserción en el notebook. Para el dataset transformado el resultado verificado es: `{'filas': 20, 'columnas': 27, 'duplicados': 0, 'nulos totales': 0, 'sin_nulos': True, 'sin duplicados': True}`.

III. Notebook ejecutable (F2)

El notebook `F2_Preprocesamiento.ipynb` constituye el núcleo operativo de la Fase 2. Se ejecuta de forma íntegra mediante «Restart & Run All» sin requerir intervención manual y sin errores, dentro del entorno reproducible configurado en la Fase 1. Su estructura es coherente con el flujo del pipeline:

1. Objetivo de la Fase 2.
2. Importación de librerías e inicialización del entorno.
3. Obtención y exploración inicial del dataset.
4. Limpieza de datos.
5. Transformación de variables.
6. Pipeline completo y persistencia.
7. Justificación técnica de las decisiones.

3.1 Evidencias de ejecución por bloque

Las siguientes capturas evidencian la ejecución verificable de cada etapa del pipeline, con resultados intermedios trazables.



Figura 3. Obtención y exploración inicial: dataset crudo (21 × 9) y perfilamiento con detección de nulos por variable.

3. Limpieza

Criterios aplicados: eliminación de duplicados, imputación de numéricas con mediana, normalización textual y relleno de categoricas no informadas.

```
df_clean = clean_dataset(df_raw)
display(df_clean.head())
display(profile_dataset(df_clean))
print(validate_dataset(df_clean))
```

	id	segmento	region	antigüedad_meses	monto_operacion	días_mora	canal	estado	score_riesgo
0	1	Empresa	Metropolitana	24.0	1500000.0	0.0	Web	Activo	0.12
1	2	Pyme	O'Higgins	12.0	850000.0	5.0	Sucursal	Activo	0.25
2	3	Empresa	Valparaíso	36.0	2300000.0	0.0	Ejecutivo	Activo	0.08
3	4	Microempresa	Metropolitana	6.0	920000.0	15.0	Web	Revision	0.44
4	5	Pyme	O'Higgins	18.0	920000.0	4.0	Sucursal	Activo	0.22

	tipo	nulos	nulos_pct	unicos
id	int64	0	0.0	20
segmento	string	0	0.0	4
region	string	0	0.0	5
antigüedad_meses	float64	0	0.0	19
monto_operacion	float64	0	0.0	19
días_mora	float64	0	0.0	13
canal	string	0	0.0	5
estado	string	0	0.0	4
score_riesgo	float64	0	0.0	19

```
{'filas': 20, 'columnas': 9, 'duplicados': 0, 'nulos_totales': 0, 'sin_nulos': True, 'sin_duplicados': True}
```

Figura 4. Limpieza: dataset depurado (20 × 9), perfilamiento sin nulos y validación sin_nulos = True, sin_duplicados = True.

1. Objetivo F2

Implementar un flujo reproducible para cargar, perfilar, limpiar, transformar y validar el dataset del proyecto.

```
import sys
from pathlib import Path
import pandas as pd
from IPython.display import display
ROOT = Path.cwd().resolve()
if ROOT.name == 'notebooks':
    ROOT = ROOT.parents[1]
sys.path.append(str(ROOT / 'F2' / 'src'))
from preprocessing import ( # noqa: E402
    load_dataset, profile_dataset, clean_dataset,
    transform_dataset, validate_dataset, run_pipeline,
)
RAW = ROOT / 'F2' / 'data' / 'raw' / 'dataset_base.csv'
OUT = ROOT / 'F2' / 'data' / 'processed' / 'dataset_procesado.csv'
print('Python:', sys.version.split()[0])
print('Pandas:', pd.__version__)
print('Dataset:', RAW)
```

```
Python: 3.12.10
Pandas: 3.0.3
Dataset: D:\LM_IA_LAB\04_PROJECTS\abp_cienciadatos\F2\data\raw\dataset_base.csv
```

Figura 5. Transformación: normalización MinMax y One-Hot Encoding; dataset resultante (20 × 27) validado sin nulos ni duplicados.

3.2 Pipeline completo y persistencia

La orquestación integral ejecuta el flujo de extremo a extremo, persiste el dataset procesado y verifica la salida mediante aserciones que detienen la ejecución ante cualquier inconsistencia:

```
clean, transformed, validation = run_pipeline(RAW, OUT)
print(validation)
assert validation['sin_nulos'], 'El dataset procesado mantiene valores nulos.'
assert validation['sin_duplicados'], 'El dataset procesado mantiene duplicados.'
assert OUT.exists(), 'No se genero el archivo procesado.'
print('Dataset procesado guardado en:', OUT)
```

Listado 2. Ejecución del pipeline completo, aserciones de calidad y persistencia del dataset procesado.

3.3 Dependencias utilizadas

El entorno reproducible declara las siguientes dependencias (gestionadas vía requirements.txt): NumPy, pandas, scikit-learn, Matplotlib, JupyterLab, Notebook, IPykernel y Pytest. La documentación integrada mediante celdas Markdown permite comprender claramente cada etapa del procesamiento.

IV. Repositorio GitHub (F2)

La organización del repositorio sigue prácticas profesionales de desarrollo reproducible y evidencia la evolución técnica del proyecto. La estructura interna es coherente con el informe y con el flujo del pipeline.

4.1 Estructura del repositorio

```
abp_cienciadatos/  
|  
+-- README.md  
+-- requirements.txt  
+-- .gitignore  
|  
+-- F1/  
+-- F2/  
|   +-- notebooks/    F2_Preprocesamiento.ipynb  
|   +-- src/          preprocessing.py  
|   +-- data/raw/      dataset_base.csv  
|   +-- data/processed/ dataset_procesado.csv  
|  
+-- calidad-aire-santiago/
```

Listado 3. Estructura del repositorio con la carpeta F2 del avance actual.

4.2 README técnico actualizado

El README incorpora la descripción del proyecto, el contenido por carpeta, el pipeline técnico de cinco etapas, las instrucciones de ejecución (macOS/Linux y Windows) y las dependencias. Las instrucciones de ejecución reproducible son:

```
# macOS / Linux  
source .venv/bin/activate  
jupyter lab F2/notebooks/F2_Preprocesamiento.ipynb  
  
# Windows (PowerShell)  
.\.venv\Scripts\Activate.ps1  
jupyter lab F2\notebooks\F2_Preprocesamiento.ipynb
```

Listado 4. Instrucciones de ejecución del notebook desde la raíz del repositorio.

4.3 Control de versiones y contribuciones individuales

Git y GitHub garantizan la trazabilidad, el historial de cambios, la colaboración grupal y la reproducibilidad. La evidencia de contribuciones individuales se mantiene visible en GitHub mediante las cuentas personales de cada integrante, con commits identificables que reflejan el progreso del pipeline de datos de esta fase. En el Anexo B se incluye la convención de commits adoptada por el equipo para asegurar un historial legible y atribuible.

V. Bibliografía (APA 7)

Las siguientes fuentes se citan explícitamente en el cuerpo del informe y en el notebook. Se presentan en orden alfabético, con títulos en cursiva, combinando documentación técnica oficial (≥ 2), bibliografía académica complementaria (≥ 1) y material docente (≥ 2).

- García, S., Luengo, J., & Herrera, F. (2015). *Data preprocessing in data mining*. Springer. <https://doi.org/10.1007/978-3-319-10247-4>
- González, M. (2025). *Guía de preprocesamiento de datos: limpieza y transformación con pandas* [Material del curso]. Magíster en Ciencia de Datos e Inteligencia Artificial, Universidad Andrés Bello.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- NumPy Developers. (2024). *NumPy documentation*. <https://numpy.org/doc/stable/>
- Pérez, J. (2025). *Obtención y exploración de datos en Python* [Presentación de clase]. Magíster en Ciencia de Datos e Inteligencia Artificial, Universidad Andrés Bello.
- Scikit-learn Developers. (2024). *Preprocessing data — scikit-learn user guide*. <https://scikit-learn.org/stable/modules/preprocessing.html>
- The pandas development team. (2024). *pandas documentation*. <https://pandas.pydata.org/docs/>

VI. Justificación técnica de las decisiones de preprocesamiento

Las decisiones de preprocesamiento implementadas en esta fase fueron seleccionadas considerando tanto las características específicas del conjunto de datos de calidad del aire para Santiago como los objetivos metodológicos del proyecto, orientados a garantizar la calidad, consistencia, trazabilidad y reproducibilidad de los datos. El propósito principal fue transformar un dataset con defectos deliberadamente incorporados —valores faltantes, registros duplicados, inconsistencias de formato y variables en escalas heterogéneas— en un conjunto de datos confiable y apto para etapas posteriores de análisis exploratorio, modelado predictivo e interpretación de resultados.

Desde una perspectiva de ciencia de datos reproducible, cada transformación aplicada fue diseñada para minimizar la introducción de sesgos artificiales, preservar la integridad estadística de las variables originales y asegurar que los resultados obtenidos puedan ser replicados de manera consistente en futuras ejecuciones del pipeline.

6.1 Imputación de valores faltantes mediante mediana

Los valores faltantes presentes en las variables numéricas fueron imputados utilizando la mediana de cada atributo. Esta decisión se fundamenta en que las variables ambientales suelen presentar distribuciones no normales, asimetrías significativas y eventos extremos asociados a episodios de alta contaminación atmosférica.

En este contexto, la media aritmética resulta especialmente sensible a valores atípicos, pudiendo desplazar artificialmente la tendencia central de los datos e introducir distorsiones en análisis posteriores. La mediana, en cambio, constituye una medida robusta de tendencia central que conserva de mejor forma la estructura estadística subyacente de la variable, reduciendo el impacto de observaciones extremas y evitando la incorporación de sesgos derivados del proceso de imputación.

La utilización de este criterio contribuye a preservar la representatividad de las mediciones ambientales y mejora la confiabilidad de los resultados obtenidos en etapas posteriores del proyecto.

6.2 Normalización de variables mediante escalamiento Min-Max

Las variables numéricas fueron transformadas mediante la técnica de escalamiento Min-Max, llevando sus valores al rango normalizado $[0,1]$.

La necesidad de esta transformación surge debido a la heterogeneidad inherente de las variables consideradas. El dataset integra mediciones expresadas en diferentes unidades y órdenes de magnitud, incluyendo concentraciones de contaminantes atmosféricos, indicadores meteorológicos y variables operacionales. Sin una normalización adecuada, los atributos con valores numéricamente más elevados podrían dominar los cálculos internos de diversos algoritmos, generando un sesgo asociado exclusivamente a la escala de medición.

El escalamiento Min-Max permite conservar la forma relativa de la distribución original de los datos, manteniendo las relaciones entre observaciones mientras homogeneiza las magnitudes. Esta propiedad resulta especialmente relevante para algoritmos sensibles a la escala, tales como K-Nearest Neighbors (KNN), Redes Neuronales Artificiales, Support Vector Machines (SVM) y métodos de optimización basados en gradiente.

Como resultado, cada variable contribuye de manera equilibrada al proceso analítico, favoreciendo la estabilidad numérica, la convergencia de los modelos y la comparabilidad entre atributos.

6.3 Transformación de variables categóricas mediante One-Hot Encoding

Las variables categóricas nominales fueron transformadas utilizando la técnica de One-Hot Encoding.

Este procedimiento se justifica porque las categorías presentes en variables como estaciones de monitoreo, zonas geográficas o clasificaciones operacionales carecen de un orden natural. Una codificación ordinal tradicional asignaría valores numéricos que podrían ser interpretados erróneamente por los algoritmos como relaciones jerárquicas o distancias entre categorías.

One-Hot Encoding elimina este riesgo al representar cada categoría mediante una variable binaria independiente. De esta forma, cada nivel mantiene su significado original sin introducir relaciones artificiales entre categorías.

Además de preservar la semántica de los datos, esta estrategia mejora la compatibilidad con algoritmos de aprendizaje automático y permite capturar adecuadamente patrones asociados a variables categóricas sin generar sesgos derivados del esquema de codificación.

6.4 Validación técnica y aseguramiento de calidad

Una vez ejecutadas las etapas de limpieza y transformación, el pipeline incorpora un proceso formal de validación orientado a verificar la calidad e integridad del conjunto de datos resultante.

Las validaciones implementadas incluyen:

- Verificación de ausencia de valores nulos.
- Verificación de eliminación completa de registros duplicados.
- Confirmación de consistencia estructural del dataset.
- Validación de tipos de datos esperados.
- Comprobación de persistencia correcta de los archivos generados.
- Ejecución de aserciones automáticas para detectar desviaciones del comportamiento esperado.

Este enfoque permite identificar tempranamente errores de procesamiento y garantiza que cada ejecución produzca resultados consistentes, verificables y trazables.

6.5 Relación con los objetivos analíticos del proyecto

Las decisiones metodológicas adoptadas durante esta fase responden directamente al objetivo estratégico del proyecto: construir un pipeline reproducible de preparación de datos que permita generar evidencia analítica confiable para el estudio de la calidad del aire en Santiago.

La combinación de imputación robusta, normalización de variables, codificación adecuada de atributos categóricos y validación sistemática permite obtener un conjunto de datos limpio, consistente y metodológicamente sólido, reduciendo la probabilidad de sesgos derivados del preprocesamiento.

Como resultado, el dataset final constituye una base confiable para el desarrollo de análisis exploratorios, construcción de modelos predictivos, identificación de patrones ambientales y generación de conocimiento útil para la toma de decisiones basada en datos.

VII. Conclusiones

La presente Fase 2 permitió diseñar, implementar y validar exitosamente un pipeline reproducible de obtención, limpieza, transformación y verificación de datos, alineado con las mejores prácticas de ingeniería de datos y ciencia de datos reproducible.

A través de una arquitectura modular basada en funciones especializadas, se logró automatizar de forma consistente cada etapa del proceso de preparación de datos, desde la carga inicial del conjunto de datos hasta la generación de un dataset final validado y persistido en disco. Este enfoque favorece la mantenibilidad, reutilización y escalabilidad de la solución, permitiendo que futuras mejoras puedan incorporarse sin afectar la trazabilidad del flujo implementado.

Los resultados obtenidos evidencian una mejora sustancial en la calidad del conjunto de datos. El pipeline transformó un dataset crudo de 21 filas y 9 columnas, que contenía registros duplicados, valores faltantes e inconsistencias de formato, en un dataset estructurado de 20 filas y 27 columnas, completamente validado, sin valores nulos, sin duplicados y con variables adecuadamente normalizadas y codificadas.

Desde una perspectiva metodológica, las decisiones adoptadas —imputación mediante mediana, escalamiento Min-Max y One-Hot Encoding— se encuentran respaldadas por criterios técnicos ampliamente aceptados en la literatura especializada y resultan coherentes con las características particulares de los datos ambientales analizados. Estas transformaciones permiten preservar la calidad estadística de la información y reducen potenciales fuentes de sesgo durante las etapas posteriores de análisis.

Asimismo, la integración de validaciones automáticas, documentación técnica detallada, control de versiones y ejecución reproducible mediante notebook fortalece significativamente la confiabilidad del proyecto, asegurando que los resultados puedan ser replicados y auditados por terceros.

En síntesis, los objetivos definidos para la Fase 2 fueron cumplidos satisfactoriamente. El proyecto dispone ahora de una base de datos depurada, consistente y técnicamente validada que constituye un insumo sólido para la Fase 3, orientada al análisis exploratorio avanzado, identificación de patrones de comportamiento de contaminantes atmosféricos y eventual construcción de modelos analíticos sobre la calidad del aire en Santiago.

La implementación realizada no solo resuelve los requerimientos académicos de la presente etapa, sino que además establece fundamentos metodológicos robustos para el desarrollo de soluciones analíticas reproducibles, escalables y alineadas con estándares profesionales de ciencia de datos.

VIII. Anexos

Anexo A. Checklist de entrega (rúbrica · 63 pts)

Criterio de rúbrica	Pts	Estado
Portada e índice	3	Cumple
Codificación funcional y arquitectura del script	9	Cumple
Preprocesamiento y transformación del dataset	9	Cumple
Validación técnica y verificación del código	9	Cumple
Preprocesamiento y transformación de datos	9	Cumple
Notebook final ejecutado y documentado	9	Cumple
Repositorio GitHub actualizado	9	Verificar commits
Aspectos formales institucionales	6	Cumple
Total	63	—

Tabla 3. Trazabilidad del informe respecto de los criterios de la rúbrica.

Nota: el criterio «Repositorio GitHub actualizado» exige evidencia de contribuciones individuales mediante commits identificables por cada integrante. Verifiquen que cada integrante (Rodrigo, Pablo y Sergio) registre al menos un commit con su cuenta personal antes de la entrega.

Anexo B. Convención de commits recomendada

Para mantener un historial legible y atribuible (Conventional Commits):

```
feat(f2):  nueva funcionalidad del pipeline
fix(f2):   correccion de defecto
docs(f2):  documentacion / README / notebook markdown
test(f2):  pruebas de validacion
chore:     mantenimiento, dependencias, .gitignore

Ejemplo:
git commit -m "feat(f2): imputacion con mediana en clean_dataset"
```

Anexo C. Comandos de validación sugeridos

```
# Reproducir el entorno
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Ejecutar el notebook de extremo a extremo
jupyter nbconvert --to notebook --execute \
    F2/notebooks/F2_Preprocesamiento.ipynb

# Verificar el dataset procesado
python -c "import pandas as pd; d=pd.read_csv('F2/data/processed/dataset_procesado.csv');
print(d.shape, d.isna().sum().sum(), d.duplicated().sum())"
```

Anexo D. Resumen de defectos del dataset crudo

Defecto	Detalle observado
Duplicados	Registro id = 12 repetido (21 filas → 20 tras depuración).

Defecto	Detalle observado
Nulos numéricos	antiguedad_meses, monto_operacion, dias_mora, score_riesgo (1 nulo c/u; 4,76 %).
Nulos categóricos	estado (1 nulo; 4,76 %).
Inconsistencia textual	« pyme », « metropolitana », « web », «EJECUTIVO» (espacios y mayúsculas).
Escalas heterogéneas	monto_operacion (cientos de miles) vs. score_riesgo (0–1): requiere normalización.

Tabla 4. Defectos deliberados del dataset base y su tratamiento en el pipeline.